

Progettazione di un'applicazione web per semplificare lo sviluppo di prodotti IoT

Christian Bonsignore

1. Introduzione

Lo sviluppo di progetti IoT richiede solitamente un lungo periodo di progettazione e implementazione vista la varietà dei componenti che costituiscono il sistema.

Infatti, in un progetto IoT si lavora alla progettazione delle componenti hardware e del firmware, la configurazione dell'infrastruttura Cloud, dallo storage dei dispositivi alle API, e lo sviluppo delle App con cui gli utenti interagiranno per utilizzare il sistema. Solo l'unione di tutti questi componenti porta ad una soluzione tecnologica completa che può essere sia un prodotto consumer, come ad esempio un articolo *Smart Home*, che uno più industriale, come nel caso di interfacce a bordo macchina. Mentre nel lavoro svolto in gruppo si forniva anche una soluzione per quanto riguarda l'**infrastruttura Cloud**, le relative API e lo sviluppo delle app mobile risultanti dal processo, questo progetto si concentra solo sulla creazione di un applicativo di configurazione utilizzabile in scenari in cui del personale tecnico specializzato deve configurare il funzionamento del sistema IoT finale in base alle esigenze del cliente.

Ad esempio questo permetterà di aggiungere entità, personalizzarne le proprietà e creare relazioni tra di loro, ma anche di configurare impostazioni relative alla sicurezza come il comportamento del sistema circa l'**autenticazione** degli utenti.

Alla fine del proprio lavoro l'utente configuratore avrà costruito una struttura per il sistema che sarebbe stata utilizzata per la generazione del codice ed archiviata in modo tale da rendere possibili future evoluzioni o revisioni del funzionamento.

Questa configurazione di sistema è costituita da un file in formato JSON che rappresenta la struttura vera e propria contenente tutte le informazioni, i parametri e le necessità del cliente riguardo alle entità e alle relazioni fra queste ultime. Questo file può anche essere completato parzialmente e poi in seguito ricaricato per essere modificato in qualunque momento.

Nell'idea originale impiegando il file JSON di configurazione si andavano a generare diversi artefatti:

- Una cartella compressa in formato ZIP contenente i file necessari al funzionamento del sistema personalizzato (**template** che descrive l'**infrastruttura cloud** e codice delle API);
- Una cartella compressa in formato ZIP contenente una parte di codice delle App mobili native per *Android/iOS*.
- Un file in formato Markdown contenente una guida passo passo per portare a termine il **deploy** del codice e delle risorse generate.

Grazie a questo processo si configura un sistema ampiamente personalizzato sulla base delle singole necessità del cliente in tempi molto più brevi rispetto all'intera procedura di creazione.

2. Disclaimer

Gran parte del progetto è stata sviluppata durante lo stage e il group project svolti presso l'azienda *DevQ S.r.l.* di Corridonia, ma in ogni caso tutto ciò che è presentato è stato fatto

da me. Le novità riguardano i seguenti aspetti:

- Introduzione di pagine di **login** e **registrazione**;
- Introduzione di **backend** e **DBMS** per gestire le richieste;
- Introduzione di misure di protezione da tecniche di **SQL Injection** e **brute force**;
- Modifiche minori ai loghi e al **routing**.

3. Tecnologie scelte

Framework: Angular;

Linguaggi di programmazione: TypeScript e Python;

Linguaggi di formattazione: HTML, CSS;

DBMS: PostgreSQL;

Software: IntelliJ, PyCharm, Visual Studio Code, Postman, pgAdmin 4;

4. Architettura del sistema

L'architettura del sistema è stata progettata seguendo il pattern **MVC**, con una chiara separazione delle responsabilità. È suddivisa in due parti principali: il **frontend**, realizzato in Angular, e il **backend**, sviluppato con **Python** e **Flask**.

4.1. Comunicazione tra Frontend e Backend

La comunicazione tra il frontend e il backend avviene tramite **API REST**, utilizzando l'HTTP per scambiare dati in formato JSON. Le richieste di **autenticazione**, **registrazione** e operazioni sulle entità vengono gestite attraverso il servizio HttpClient di Angular, che invia le richieste al backend e gestisce le risposte.

4.2. Sicurezza

Per garantire la **sicurezza** del sistema, sono stati implementati controlli per prevenire attacchi come **SQL Injection**. Inoltre, il servizio di autenticazione gestisce la sessione dell'utente tramite token **JWT**, assicurando che solo gli utenti autenticati possano accedere a funzionalità sensibili.

5. Frontend

Il frontend dell'applicazione è una **Single Page Application** progettata in Angular, con una struttura suddivisa fra componenti e servizi.

Questi i componenti:

Login e Register: Gestiscono le funzionalità di registrazione e login degli utenti;

Homepage: La prima pagina del sito da cui scegliere se iniziare la configurazione per un nuovo progetto o riprenderne una scaricata in precedenza;

Drawer: Gestisce il contenuto del drawer e la visualizzazione *real time* del file JSON risultante;

Diagram: Gestisce il nucleo del progetto, tutte le funzionalità relative all'interazione con il grafico. Si interfaccia costantemente con il **ConfigurationService** per mantenere una configurazione sempre coerente alle modifiche attuate;

Header: Rappresenta l'header della pagina di interazione con il diagramma, contiene la possibilità di modificare il titolo;

Workspace: Raggruppa i componenti Drawer, Diagram e Header in una singola pagina.

I servizi gestiscono la logica applicativa e le operazioni di stato, come la gestione delle entità e l'esportazione della configurazione in formato JSON.

Centrale per il progetto l'utilizzo della libreria per Angular **DevExtreme**, che fornisce molti dei componenti utilizzati per il progetto con un altissimo grado di personalizzazione, perfetta quindi per soddisfare le necessità specifiche del contesto.

6. Backend e DBMS

Il backend è implementato utilizzando **Flask** e **SQLAlchemy** per la gestione della comunicazione con il database **PostgreSQL**.

API REST: sono disponibili endpoint RESTful per gestire la registrazione e il login degli utenti. Vengono utilizzati **JSON Web Tokens (JWT)** per autenticare le richieste e garantire che solo gli utenti autenticati possano accedere a determinate risorse.

Gestione del Database: Per l'interazione con il database è stato utilizzato **SQLAlchemy**, garantendo così un'architettura dei dati robusta e sicura. I modelli del database sono definiti per gestire le entità e le relazioni necessarie per il sistema.